

Homework 2 Written Question Answers

Problem 1b

- **input_ids**: These are numerical representations of tokens after being tokenized by BERT's tokenizer. Each token has its unique id in the tokenizer's vocabulary. The input_ids will be an ordered list corresponding to the order of tokens in the original text.
- **token_type_ids**: They tell which segment/sequence each token belongs to. Tokens from the first sequence will be 0s, those from the second sequence will be 1s. It gives a list of 1/0 of the same order of tokens in the original text. Particularly, padding tokens will be assigned to 0.
- **attention_mask**: They tell which token the model should pay attention to. Each token is assigned a binary value (0 or 1) based on whether it is a padding token or not. 1 means the token should be attended to, and 0 means it should not.

Problem 1c

For hyperparameter tuning, it focuses on batch size and learning rate, while keeping the number of epochs fixed at 4. The batch size is selected from [8, 16, 32, 64, 128], and the learning rate is chosen from [3e-4, 1e-4, 5e-5, 3e-5]. Since GLUE comprises multiple evaluation metrics, it aims to identify the optimal hyperparameters for each specific task rather than applying a single configuration across all tasks.

To achieve this, we could use GridSampler for an exhaustive search across all hyperparameter combinations, ensuring fair evaluation by maintaining consistency in data splits and cross-validation strategy. The best-performing combination for each task is selected based on evaluation results. Once identified, the optimal hyperparameters are then used to fine-tune the model on the test set, and the final results are reported in a summary table.

Problem 3a

The results are shown in the following table:

	Validation Accuracy	Learning Rate	Batch Size
Without BitFit	0.8896	0.0001	8
With BitFit	0.6346	0.0003	16

Problem 3b

The results are shown in the following table:

	# Trainable Parameters	Test Accuracy
Without BitFit	4385920	0.8776
With BitFit	3087	0.6357

Comments: The Bifit method does not perform as well as suggested in the original paper. Both validation and test accuracy drop significantly when using Bifit, which is expected since it involves far fewer trainable parameters, limiting optimization flexibility. Empirically, the results do not support the claim that Bifit outperforms full model fine-tuning. However, Bifit remains a promising approach due to its reduced training time and resource requirements. If achieving the best possible accuracy (e.g., classification performance) is the priority, fine-tuning the entire model is still the preferable choice.